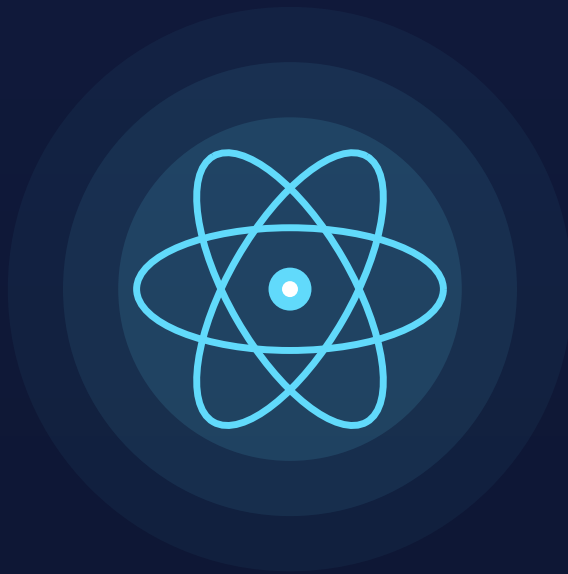




React Interview Mastery Guide

159 Questions, Answers, Explanations & Code

Study it once. Walk into any React interview ready.

JavaScript

Beginner

Intermediate

Advanced

Output-Based

Coding Round

HR

CONNECT WITH THE AUTHOR



LinkedIn



GitHub

Covers React 18 & React 19 - Hooks, Performance, SSR, RSC, Redux, Testing

47 pages | 6 difficulty levels | 20 coding-round problems

Table of Contents

How to Use This Guide	3
Level 0 - JavaScript Essentials for React Interviews	4
Level 1 - React Fundamentals (Beginner / Fresher)	8
Level 2 - Intermediate React (1-3 Years Experience)	15
Level 3 - Advanced React (3+ Years / Senior)	24
Level 4 - Tricky and Output-Based Questions	32
Level 5 - Machine Coding / Live Coding Round	36
Level 6 - Behavioural, Project and HR Questions	43
Rapid-Fire Questions (Answer in One Line)	45
Quick Reference Cheat Sheet	46

How to Use This Guide

This guide follows the order in which real React interviews progress. Start at Level 0 and move up. For every question: (1) cover the answer and try to say it aloud, (2) read the answer, (3) read the explanation - it tells you what the interviewer is really testing, (4) type the code yourself without copying.

- **Fresher / 0-1 year:** Levels 0, 1, 2 (first half), 4, 5, 6.
- **1-3 years:** Levels 1-5 completely, plus the mock checklist.
- **3+ years / Senior:** Focus on Levels 2-5 and architecture, performance, security and system design.
- **Machine-coding rounds:** Practise every problem in Level 5 with a timer (30 minutes each).

4-Week Study Plan

Week	Focus	Outcome
1	JavaScript essentials (closures, async, ES6), React basics: JSX, props, state, events, lists, forms, useEffect	Build Todo + Weather app; answer all Level 0-1 questions
2	Hooks in depth (useRef, useReducer, Context, useMemo/useCallback), custom hooks, routing, data fetching	Build a multi-page app with React Router + API + auth; finish Level 2
3	State management (Redux Toolkit/Zustand), TanStack Query, performance, testing, TypeScript basics	Add global state and tests to your project; profile with DevTools
4	Advanced (Fiber, concurrent features, SSR/RSC, React 19), coding round practice, mock interviews	Solve all Level 5 problems; record yourself answering Level 3 and 6

Golden Rules for Answering in Interviews

- Start with a one-line definition, then explain how it works, then give a use case or short example.
- Mention trade-offs and when NOT to use something - this signals seniority.
- If you do not know, say what you do know and how you would find out; never bluff.
- Think aloud while coding, ask clarifying questions, handle loading/error/empty states.
- Keep your own projects ready as examples for almost every question.

Level 0 - JavaScript Essentials for React Interviews

Every React interview starts with JavaScript. If you are weak here, React answers will fall apart. Master these first.

Q1. Explain closures with an example. Why do they matter in React?

ANSWER

A closure is a function that remembers the variables from the scope in which it was created, even after that outer scope has finished executing.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

In React every render creates new function instances that close over that render's props and state. This is the root cause of the famous 'stale closure' bug in `useEffect` and `setInterval`, and it is also why event handlers 'see' the state of the render they were created in.

CODE

```
function counter() {  
  let count = 0;  
  return () => ++count;  // inner fn closes over count  
}  
const inc = counter();  
inc(); // 1  
inc(); // 2
```

Q2. var vs let vs const? What is hoisting and the Temporal Dead Zone?

ANSWER

- **var**: function-scoped, hoisted and initialised as **undefined**, can be redeclared.
- **let**: block-scoped, hoisted but NOT initialised (TDZ), cannot be redeclared.
- **const**: block-scoped like let, must be initialised, binding cannot be reassigned (the object it points to can still be mutated).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

The Temporal Dead Zone is the time between entering the scope and the line where a `let/const` is declared; touching the variable there throws a `ReferenceError`. In React always use `const` by default - it signals that the binding is not reassigned, but remember `const` does not make state immutable.

CODE

```
console.log(a); // undefined (var hoisted)  
console.log(b); // ReferenceError (TDZ)  
var a = 1;  
let b = 2;
```

Q3. How does **this** work? How do arrow functions differ?

ANSWER

this is decided by HOW a function is called (call-site), not where it is defined: method call -> the object, plain call -> undefined (strict mode) / window, **new** -> the new object, `call/apply/bind` -> the explicit value. Arrow functions have no own **this**; they capture it lexically from the surrounding scope.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

This is why class components needed `this.handleClick = this.handleClick.bind(this)` (or arrow class fields), and why function components with hooks avoid the whole problem.

Q4. Explain the JavaScript event loop, call stack, microtasks and macrotasks.

ANSWER

JS is single-threaded. The call stack runs synchronous code. Async work (timers, network, DOM events) is handled by browser APIs, and when done, callbacks are queued. The event loop moves queued callbacks to the stack only when the stack is empty. Microtasks (Promise.then, queueMicrotask, MutationObserver) always run before the next macrotask (setTimeout, setInterval, I/O, UI events).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Classic output question. It also explains why React state updates inside setTimeout/promises used to render synchronously (pre-18) and why useEffect runs after paint.

CODE

```
console.log(1);
setTimeout(() => console.log(2), 0);
Promise.resolve().then(() => console.log(3));
console.log(4);
// Output: 1 4 3 2
```

Q5. Promises vs async/await. How do you handle errors?

ANSWER

A Promise represents a future value with states pending, fulfilled or rejected. `async/await` is syntax sugar over promises that makes async code read sequentially. Handle errors with `.catch()` or try/catch around await. Use `Promise.all` for parallel work, `Promise.allSettled` when you need every result, `Promise.race` for the first to settle.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

In React, remember you cannot make the useEffect callback itself async (it must return nothing or a cleanup function). Define an async function inside and call it.

CODE

```
useEffect(() => {
  async function load() {
    try {
      const res = await fetch('/api/users');
      setUsers(await res.json());
    } catch (err) { setError(err); }
  }
  load();
}, []);
```

Q6. Destructuring, spread and rest - where do you use them in React?

ANSWER

Destructuring unpacks values from arrays/objects into variables. Spread (...) expands an iterable/object; rest (...) collects the remaining items.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

React uses them constantly: `const [count, setCount] = useState(0)`, `function Card({title, ...rest})`, and immutable updates `setUser({...user, name: 'A'})` or `setList([...list, item])`.

CODE

```
const [items, setItems] = useState([]);
setItems(prev => [...prev, newItem]); // add
setItems(prev => prev.filter(i => i.id !== id)); // remove
setUser(prev => ({ ...prev, age: 21 })); // update
```

Q7. map, filter, reduce, find, some, every - explain and give a use case.

ANSWER

- `map`: transform each item, returns a new array (render lists).
- `filter`: keep items that pass a test.

- **reduce**: fold array into a single value (sum, group by).
- **find**: first matching item; **some/every**: boolean checks.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

None of them mutate the original array - this is exactly why they are safe to use for React state updates. **sort**, **push**, **splice**, **reverse** DO mutate; copy first (`[...arr].sort()` or `toSorted()`).

Q8. == vs === ? What are truthy and falsy values?

ANSWER

`==` compares after type coercion, `===` compares value and type (strict). Falsy values: **false**, **0**, **-0**, **0n**, **''**, **null**, **undefined**, **NaN**. Everything else is truthy (including `[]`, `{}` and `'0'`).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Always use `===`. Falsy knowledge matters in JSX: `{count} && <List/>` renders the number 0 when count is 0.

Q9. Shallow copy vs deep copy? How do you copy objects safely?

ANSWER

A shallow copy (`spread`, `Object.assign`, `slice`) copies only the top level; nested objects are still shared by reference. A deep copy duplicates everything: `structuredClone(obj)` (modern, built-in) or `JSON.parse(JSON.stringify(obj))` (loses functions, Dates, undefined).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

React compares state by reference (`Object.is`). Shallow copying the changed path is enough for React to notice a change - you only need to copy the objects on the path you are changing, not the entire tree.

Q10. Explain call, apply and bind.

ANSWER

All three set **this** explicitly. `call(thisArg, a, b)` invokes immediately with arguments listed, `apply(thisArg, [a, b])` invokes immediately with an array, `bind(thisArg, a)` returns a NEW function with **this** (and optional arguments) fixed.

Q11. What is prototypal inheritance? What does a JS class actually do?

ANSWER

Every object has an internal `[[Prototype]]` link. When a property is not found on the object, JS looks up the prototype chain. ES6 **class** is syntactic sugar over constructor functions and prototypes.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Relevant because React class components use `extends React.Component`, and it explains how methods like `setState` are inherited.

Q12. What are debounce and throttle?

ANSWER

Debounce delays execution until the user stops triggering the event for N ms (search box). Throttle guarantees the function runs at most once every N ms (scroll, resize).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Very common machine-coding question. Full implementations appear in the coding section of this PDF.

Q13. What are ES modules? Named vs default export?

ANSWER

ES modules use **import/export** and are statically analysable (enables tree-shaking). A file can have many named exports (`import {a} from`) but only one default export (`import x from`).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

React.lazy needs a module with a default export. Named exports are generally preferred for better auto-import and refactoring.

Q14. Optional chaining, nullish coalescing and short-circuiting?**ANSWER**

`a?.b` returns undefined instead of throwing if `a` is null/undefined. `a ?? b` uses `b` only when `a` is null/undefined (unlike `||`, which also replaces 0 and ""). `&&` and `||` short-circuit.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Use `user?.address?.city` when rendering API data that may not have loaded yet.

Q15. Explain the CSS box model, Flexbox vs Grid, and the difference between em, rem, % and vh.**ANSWER**

Box model: content + padding + border + margin (`box-sizing: border-box` includes padding and border in width). Flexbox is one-dimensional (row or column), Grid is two-dimensional. `rem` is relative to root font size, `em` to the parent's font size, `%` to the parent's dimension, `vh` to 1% of viewport height.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

React roles almost always include one CSS question (center a div, responsive layout). Practise: flex centering, grid with `repeat(auto-fit, minmax())`, position absolute/relative, media queries.

Level 1 - React Fundamentals (Beginner / Fresher)

These questions are asked in almost every interview, especially for freshers and 0-2 years experience. You must answer these confidently without hesitation.

Q16. What is React and why is it used?

ANSWER

React is an open-source JavaScript LIBRARY (created by Facebook/Meta) for building user interfaces using reusable components. It is declarative: you describe what the UI should look like for a given state and React figures out how to update the DOM.

Key benefits:

- Component-based reusability
- Virtual DOM for efficient updates
- Unidirectional data flow (predictable)
- Large ecosystem and community
- Same knowledge reusable for mobile via React Native

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Say 'library, not framework' - React only handles the view layer. Routing (React Router), data fetching (TanStack Query), state at scale (Redux/Zustand) and SSR (Next.js) come from the ecosystem.

Q17. What is JSX? Can browsers read it?

ANSWER

JSX is a syntax extension that lets you write HTML-like markup inside JavaScript. Browsers cannot read it; a compiler (Babel, SWC, esbuild) converts it to function calls (`React.createElement` or the automatic `jsx()` runtime).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

JSX rules: return a single root (or a Fragment), close every tag (`<img />`), use camelCase attributes (`className`, `htmlFor`, `onClick`), embed JS expressions in `{}` (expressions only, not statements like `if/for`).

CODE

```
const el = <h1 className="title">Hello</h1>;
// compiles to:
const el = React.createElement('h1', { className: 'title' }, 'Hello');
```

Q18. What is the difference between an element and a component?

ANSWER

An element is a plain, immutable JavaScript object describing what to render, e.g. `{ type: 'div', props: {...} }`. A component is a function (or class) that accepts props and returns elements.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Components are blueprints; elements are the descriptions they produce. `<Button />` creates an element whose `type` is the `Button` function.

Q19. Functional vs class components?

ANSWER

Function components are plain functions returning JSX and use Hooks for state and side effects. Class components extend `React.Component`, use `this.state`, `this.setState` and lifecycle methods (`componentDidMount` etc.).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Since React 16.8 function components are the standard: less boilerplate, no `this` binding, and logic is shareable via custom hooks. Classes are still supported; Error Boundaries still require a class component (no hook equivalent yet).

Q20. Props vs state - what is the difference?

ANSWER

- Props: inputs passed from parent to child; read-only inside the child.
- State: data owned and managed inside the component; changing it triggers a re-render.
- Props are like function arguments, state is like variables inside the function that React remembers.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

A child must never modify its props. If a child needs to change parent data, the parent passes down a callback function as a prop (lifting state up).

Q21. What is the Virtual DOM and how does it work?

ANSWER

The Virtual DOM is a lightweight JavaScript representation (tree of objects) of the real DOM. When state changes, React builds a new virtual tree, compares it with the previous one (diffing), calculates the minimum set of changes and applies only those to the real DOM (reconciliation/commit).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Real DOM operations (layout, paint) are expensive, JS object comparisons are cheap. Important nuance: the Virtual DOM is not 'faster than the DOM' - it makes declarative UI code efficient enough. Svelte and Solid skip it entirely.

Q22. What is reconciliation and how does the diffing algorithm work?

ANSWER

Reconciliation is the process of comparing the new element tree with the old one and updating the DOM. React uses two heuristics to get $O(n)$ instead of $O(n^3)$:

1. Elements of different types produce different trees (React tears down the old subtree and builds a new one).
2. Developers can hint stable identity across renders using the `key` prop.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Same type at same position: React keeps the DOM node and component state and only updates changed props.
Different type: unmount old (state lost), mount new.

Q23. Why do we need keys in lists? Can we use index as key?

ANSWER

Keys give each list item a stable identity so React can match items between renders and correctly add, remove or reorder them without recreating everything. Keys must be unique among siblings and stable (ids from your data).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Index as key is acceptable ONLY for static lists that never reorder, filter or insert. Otherwise it causes wrong state to stick to wrong items (typed input text jumps to another row) and unnecessary re-mounts. Never use `Math.random()` as key - it remounts every render.

CODE

```
{todos.map(todo => (  
  <li key={todo.id}>{todo.text}</li>  
))}
```

Q24. Controlled vs uncontrolled components?

ANSWER

Controlled: form value lives in React state and the input displays `value={state}` with an `onChange` handler - React is the single source of truth. Uncontrolled: the DOM holds the value and you read it with a ref when needed (`ref.current.value`) or FormData.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Use controlled for validation, instant feedback, dependent fields. Use uncontrolled for simple forms and file inputs (always uncontrolled) or when performance matters; libraries like React Hook Form use uncontrolled inputs for speed.

CODE

```
const [name, setName] = useState('');

```

Q25. What are Fragments and why are they used?

ANSWER

A Fragment (`<React.Fragment>` or `<>...</>`) lets you group multiple elements without adding an extra DOM node.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Useful for table rows (`<tr>` inside a `<div>` is invalid HTML) and to keep DOM clean. Only the long form `<Fragment key={id}>` accepts a key.

Q26. How do you do conditional rendering in React?

ANSWER

- if/else or early return in the component
- Ternary: `{isLoggedIn ? <Dashboard/> : <Login/>}`
- Logical AND: `{isAdmin && <AdminPanel/>}`
- Return `null` to render nothing
- Switch/object map for many cases

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Gotcha: `{items.length && <List/>}` renders the number 0 when empty. Use `items.length > 0 &&` or a ternary.

Q27. How are events handled in React? What is a SyntheticEvent?

ANSWER

React events are named in camelCase and you pass a function, not a string: `onClick={handleClick}`. React wraps the native event in a SyntheticEvent - a cross-browser wrapper with the same interface (`preventDefault`, `stopPropagation`, `target`).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

React attaches handlers at the root container (event delegation) rather than on each node. Since React 17 the root container is used instead of `document`. Common mistake: `onClick={handle()}` calls the function during render; use `onClick={handle}` or `onClick={() => handle(id)}`.

Q28. What is useState? How does it work?

ANSWER

`useState(initial)` returns `[value, setValue]`. React stores state per component instance, outside the function. Calling the setter schedules a re-render; on the next render the hook returns the new value.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Rules: never mutate state directly; state updates are batched and apply on the NEXT render (the variable in the current render does not change). When new state depends on old state use the functional form `setCount(c => c + 1)`.

CODE

```
const [count, setCount] = useState(0);
<button onClick={() => setCount(c => c + 1)}>{count}</button>
```

Q29. What is useEffect? Explain the dependency array.**ANSWER**

`useEffect` runs side effects (data fetching, subscriptions, timers, manual DOM work) after the component renders and the browser paints.

- No array: runs after every render.
- `[]`: runs once after the first mount.
- `[a, b]`: runs after mount and whenever a or b change.

The function can return a cleanup which runs before the next effect and on unmount.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Think of it as 'synchronise this component with something outside React whenever these values change', not as `componentDidMount`.

CODE

```
useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id); // cleanup
}, []);
```

Q30. What is props drilling and how do you avoid it?**ANSWER**

Passing props through many intermediate components that do not need them just to reach a deep child.

Solutions: Context API, state-management libraries (Redux Toolkit, Zustand), component composition (pass children/elements instead of data), custom hooks.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Prefer composition first - e.g. pass `<Avatar user={user}/>` as a prop/children instead of drilling `user` through 4 layers. Reach for Context only for truly global data (theme, auth, locale).

Q31. What is lifting state up?**ANSWER**

When two sibling components need the same data, move the state to their closest common parent and pass it down as props, along with setter callbacks.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

This keeps a single source of truth and is React's default approach before reaching for global state.

Q32. What is the children prop?**ANSWER**

A special prop containing whatever is placed between a component's opening and closing tags. It enables composition and wrapper components (Card, Layout, Modal).

CODE

```
function Card({ children }) {
  return <div className="card">{children}</div>;
}
<Card><h2>Hi</h2></Card>
```

Q33. Why should state not be mutated directly?

ANSWER

React detects changes by comparing references (Object.is). If you mutate an object/array in place and pass the same reference, React thinks nothing changed and skips the re-render. It also breaks memoization (`React.memo`, `useMemo`) and time-travel debugging.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Always create new copies: spread, map, filter, or use Immer.

CODE

```
// WRONG
todos.push(newTodo); setTodos(todos);
// RIGHT
setTodos([...todos, newTodo]);
```

Q34. What is StrictMode?

ANSWER

`<React.StrictMode>` is a development-only wrapper that highlights potential problems: it double-invokes render functions, state initialisers and effects (mount -> unmount -> mount) to expose impure code and missing cleanups, and warns about deprecated APIs.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

That is why your `useEffect` logs twice in development. It does not happen in production. It is a feature, not a bug - fix the effect so it is resilient to remounting.

Q35. What are the Rules of Hooks?

ANSWER

1. Call hooks only at the top level of a component or custom hook - never inside loops, conditions or nested functions.
2. Call hooks only from React function components or custom hooks (names starting with `use`).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

React tracks hooks by call ORDER in each render. Conditional hooks change the order and corrupt state. The `eslint-plugin-react-hooks` rule enforces this.

Q36. What is one-way data flow?

ANSWER

Data flows in a single direction - from parent to child via props. Children communicate upward only through callbacks. This makes state changes predictable and easy to debug.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Two-way binding (Angular `ngModel`) hides updates; React makes them explicit with `value + onChange`.

Q37. How do you handle forms in React?

ANSWER

Use controlled inputs with state (one state object for the form), a generic change handler using `name` attribute, and `onSubmit` with `e.preventDefault()`. For big forms use React Hook Form or Formik with schema validation (Zod/Yup).

CODE

```
const [form, setForm] = useState({ email: '', pass: '' });
const onChange = e =>
  setForm({ ...form, [e.target.name]: e.target.value });
const onSubmit = e => { e.preventDefault(); login(form); };
```

Q38. What triggers a re-render in React?

ANSWER

1. The component's own state changes (setState).
2. Its parent re-renders (children re-render by default, even if props are unchanged).
3. A context it consumes changes value.
4. A custom hook it uses (store subscription) triggers an update.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Props changing is NOT itself a trigger - the parent re-rendering is. Re-render does not mean DOM update; React re-runs your function, diffs, and touches the DOM only if output differs.

Q39. What is a pure component / pure function in React?

ANSWER

A component is pure if, for the same props, state and context, it always returns the same JSX and causes no side effects during render. React assumes purity so it can re-run, pause or skip renders.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Never mutate variables outside the component, call fetch, or modify the DOM during render. Side effects belong in event handlers or useEffect.

Q40. How do you render a list and what is the role of map()?

ANSWER

Use `array.map()` to convert data into an array of JSX elements, each with a unique `key`. React renders arrays of elements natively.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Always handle the empty state (show 'No results') and loading state, not only the happy path.

Q41. React vs Angular vs Vue - what is the difference?

ANSWER

- React: UI library, JSX, one-way flow, huge ecosystem, flexible.
- Angular: full framework (TypeScript, DI, RxJS, router, forms built in), opinionated.
- Vue: progressive framework, template syntax, reactivity system, gentle learning curve.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Give a balanced answer. Interviewers check that you know trade-offs, not that you badmouth others.

Q42. What is Vite/CRA and what happens when you run npm start?

ANSWER

They are project tooling. Create React App (deprecated in 2025) used Webpack. Vite uses native ES modules and esbuild for a very fast dev server with instant hot module replacement, and Rollup for production builds. The dev server compiles JSX, serves modules and reloads the browser on change.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Today the official recommendation is to start with a framework (Next.js, React Router framework mode) or Vite for a plain SPA.

Q43. What are default props and PropTypes? What is the modern alternative?

ANSWER

PropTypes validate prop types at runtime in development. Default values are now set with default parameters in destructuring (`function Btn({size = 'md'})`); `defaultProps` on function components is deprecated. TypeScript is the modern approach for type-safety at build time.

CODE

```
type ButtonProps = { label: string; onClick: () => void; size?: 'sm' | 'md' };  
function Button({ label, onClick, size = 'md' }: ButtonProps) {...}
```

Q44. What is the difference between the real DOM and Shadow DOM?**ANSWER**

The Virtual DOM is a React concept (an in-memory object tree used for diffing). The Shadow DOM is a browser feature (part of Web Components) that gives encapsulated DOM and style scope to an element. They are unrelated.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

A very common confusion - being able to separate the three terms (real, virtual, shadow) scores easy points.

Level 2 - Intermediate React (1-3 Years Experience)

Hooks in depth, performance, patterns, routing, state management and data fetching. This is where most interviews are decided.

Q45. What is useRef and what are its use cases?

ANSWER

`useRef(initial)` returns a mutable object `{ current }` that persists across renders. Changing `.current` does NOT trigger a re-render.

Use cases:

- Accessing DOM nodes (focus an input, measure size, scroll)
- Storing mutable values that should not cause re-render (timer ids, previous value, latest callback)
- Tracking whether component is mounted

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

State = re-render on change; ref = no re-render. Never read or write `ref.current` during rendering (except lazy init) because it makes render impure.

CODE

```
const inputRef = useRef(null);
useEffect(() => { inputRef.current.focus(); }, []);
return <input ref={inputRef} />;
```

Q46. Explain Context API and useContext. What are its drawbacks?

ANSWER

Context lets a parent provide a value to any descendant without passing props manually. Steps:

`createContext(default)`, wrap tree with `<Ctx.Provider value={...}>`, read with `useContext(Ctx)`.

Drawback: EVERY consumer re-renders whenever the provider value changes (no selector support). A new object literal as value creates re-renders on every parent render.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Mitigations: memoise the value with `useMemo`, split contexts (state vs dispatch, or by domain), keep frequently changing data out of context, or use a store like Zustand/Redux with selectors. Context is a dependency-injection mechanism, not a full state-management tool.

CODE

```
const ThemeCtx = createContext('light');
function App() {
  const [theme, setTheme] = useState('light');
  const value = useMemo(() => ({ theme, setTheme }), [theme]);
  return <ThemeCtx.Provider value={value}><Page/></ThemeCtx.Provider>;
}
const { theme } = useContext(ThemeCtx);
```

Q47. What is useReducer? When should you prefer it over useState?

ANSWER

`useReducer(reducer, initialState)` returns `[state, dispatch]`. State changes happen by dispatching actions to a pure reducer `(state, action) => newState`.

Prefer it when: state has multiple sub-values that change together, next state depends on complex logic, many actions update the same state, or you want to move logic out of components for testing.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

It is Redux's model at component level. Combine with Context to get a lightweight global store. `dispatch` identity is stable, so passing it down does not break memoisation.

CODE

```
function reducer(state, action) {
  switch (action.type) {
    case 'inc': return { count: state.count + 1 };
    case 'reset': return { count: 0 };
    default: throw new Error('Unknown action');
  }
}
const [state, dispatch] = useReducer(reducer, { count: 0 });
```

Q48. useMemo vs useCallback vs React.memo - explain the difference.

ANSWER

- `useMemo(fn, deps)` caches a computed VALUE between renders.
- `useCallback(fn, deps)` caches a FUNCTION reference (equal to `useMemo(() => fn, deps)`).
- `React.memo(Component)` caches a component's rendered output and skips re-render if props are shallow-equal.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

`useCallback` is only useful when the function is passed to a memoised child (`React.memo`) or used as an effect dependency. Otherwise it adds cost with no benefit. Measure with the Profiler before optimising.

CODE

```
const total = useMemo(() => items.reduce(sum, 0), [items]);
const onSelect = useCallback(id => setSel(id), []);
const Row = React.memo(function Row({ item, onSelect }) {...});
```

Q49. What is React.memo and when does it NOT work?

ANSWER

`React.memo` is a HOC that skips re-rendering a component when its props are shallowly equal to the previous props.

It fails when you pass new references each render: inline objects `style={{a:1}}`, inline arrow functions, arrays created in render, or children JSX (new element each time).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Fix by memoising those props (`useMemo/useCallback`), moving constants outside the component, or restructuring so the changing part is isolated. You can provide a custom comparison function as 2nd argument.

Q50. What are custom hooks? Why and how do you create one?

ANSWER

A custom hook is a JavaScript function whose name starts with `use` and that calls other hooks. It extracts reusable stateful logic (not UI) so multiple components can share it.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Each component calling the hook gets its OWN isolated state - hooks share logic, not state. Examples: `useFetch`, `useLocalStorage`, `useDebounce`, `useWindowSize`, `useOnClickOutside`.

CODE

```
function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = useCallback(() => setOn(o => !o), []);
  return [on, toggle];
}
```

Q51. Explain useEffect cleanup. Map class lifecycle methods to hooks.

ANSWER

The function returned from an effect is the cleanup; it runs before the effect re-runs and when the component unmounts. Use it to clear timers, unsubscribe listeners, cancel requests.

- constructor/initial state -> useState
- componentDidMount -> useEffect(fn, [])
- componentDidUpdate -> useEffect(fn, [deps])
- componentWillUnmount -> cleanup function
- shouldComponentUpdate -> React.memo
- getDerivedStateFromError/componentDidCatch -> still needs class (Error Boundary)

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Do not think in lifecycles; think in synchronisation: each effect starts a synchronisation process and the cleanup stops it.

Q52. useEffect vs useLayoutEffect?

ANSWER

`useEffect` runs asynchronously AFTER the browser has painted. `useLayoutEffect` runs synchronously after DOM mutations but BEFORE paint and blocks painting.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Use `useLayoutEffect` only for measuring layout / synchronously adjusting DOM to avoid flicker (tooltip position, scroll restoration). It does not run on the server (SSR warning). Default to `useEffect`.

Q53. Why does my useEffect run in an infinite loop?

ANSWER

Typical causes: (1) setting state inside an effect that has no dependency array, (2) an object/array/function created during render placed in the dependency array (new reference every render), (3) the effect updates a value that is itself in its dependencies.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Fixes: add the right dependency array, memoise objects/functions (`useMemo/useCallback`), depend on primitive values, move constants outside the component, or use the functional `setState` form so you can remove state from deps.

CODE

```
// BUG: options is a new object each render
useEffect(() => { fetchData(options); }, [options]);
// FIX: depend on primitives
useEffect(() => { fetchData({ page, q }); }, [page, q]);
```

Q54. What is a stale closure in React and how do you fix it?

ANSWER

A stale closure happens when a function (effect, timeout, event handler) captures state/props from an old render and keeps using outdated values.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Fixes: add the variable to the dependency array, use the functional updater form (`setCount(c => c + 1)`), or store the latest value in a ref. Classic case is `setInterval` with an empty dependency array reading `count`.

CODE

```
useEffect(() => {
  const id = setInterval(() => {
    setCount(c => c + 1); // not setCount(count + 1)
  }, 1000);
  return () => clearInterval(id);
}, []);
```

Q55. Is `setState` synchronous or asynchronous? What is batching?

ANSWER

State updates are scheduled, not applied immediately. The state variable in the current render stays the same; the new value shows up on the next render. React batches multiple `setState` calls in the same event into a single re-render. Since React 18 this 'automatic batching' also applies inside promises, timeouts and native event handlers.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

To force a synchronous flush (rare) use `flushSync` from `react-dom`.

CODE

```
function handle() {
  setCount(count + 1);
  setCount(count + 1);
  console.log(count); // still the OLD value
} // final count = old + 1, not + 2
```

Q56. What are Portals? Give a use case.

ANSWER

`ReactDOM.createPortal(children, domNode)` renders children into a different DOM node outside the parent's hierarchy, while keeping React tree behaviour (context, event bubbling).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Use for modals, tooltips, dropdowns and toasts that must escape `overflow: hidden` or z-index stacking contexts of their parents.

Q57. What are Error Boundaries? What errors do they not catch?

ANSWER

Error Boundaries are class components implementing `static getDerivedStateFromError` and/or `componentDidCatch`. They catch errors during rendering, lifecycle methods and constructors of the child tree and display a fallback UI instead of crashing the entire app.

They do NOT catch: event handler errors, async code (`setTimeout`, promises), server-side rendering errors, or errors thrown in the boundary itself.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

In practice use the `react-error-boundary` library (gives hooks like `useErrorBoundary` and `reset`). Place boundaries around route-level and widget-level sections; also log to Sentry.

Q58. What is code splitting, `React.lazy` and `Suspense`?

ANSWER

Code splitting divides the bundle into chunks loaded on demand so the initial load is smaller. `React.lazy(() => import('./Page'))` dynamically loads a component; `<Suspense fallback={<Spinner/>}>` shows the fallback while it loads.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Split at route level first (biggest win), then heavy widgets (charts, editors, modals). `Suspense` also handles data-loading in modern frameworks.

CODE

```
const Dashboard = lazy(() => import('./Dashboard'));
<Suspense fallback={<Spinner />}>
  <Dashboard />
</Suspense>
```

Q59. What is a Higher-Order Component (HOC)?

ANSWER

A function that takes a component and returns a new enhanced component: `const Enhanced = withAuth(Component)`. Used for cross-cutting concerns (auth, logging, theming). Examples: `connect` in old Redux, `React.memo`.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Hooks replaced most HOCs because they avoid wrapper hell and prop-name collisions. Remember to copy statics, forward refs and not create HOCs inside render.

Q60. What is the render props pattern?

ANSWER

A component receives a function prop that returns JSX and calls it with its internal data, letting the caller decide what to render: `<Mouse render={pos => <Cat pos={pos}/>} />`.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Mostly replaced by custom hooks but still seen in libraries (Formik, React Router legacy, Downshift). `children` as function is the same idea.

Q61. What is the compound component pattern?

ANSWER

A set of components that work together sharing implicit state via Context, giving consumers flexible composition: `<Tabs><Tabs.List/><Tabs.Panel/></Tabs>`. Examples: Radix UI, Headless UI, `<select><option>`.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

It demonstrates API design skills: flexible, declarative, avoids prop explosion.

Q62. What is forwardRef? What changed in React 19?

ANSWER

By default a function component cannot receive `ref` as a prop. `forwardRef` wraps a component to pass a ref down to a DOM node or child. In React 19 `ref` is a regular prop for function components, so `forwardRef` is no longer needed (and will be deprecated).

CODE

```
// React 18
const Input = forwardRef((props, ref) => <input ref={ref} {...props}/>);
// React 19
function Input({ ref, ...props }) { return <input ref={ref} {...props}/>; }
```

Q63. Explain React Router v6 basics: Routes, Link, useNavigate, params, nested routes, protected routes.

ANSWER

- `BrowserRouter` enables client-side routing using the History API.
- `<Routes>/<Route path element>` define routes; v6 matches by best fit (no `exact`).
- `<Link>/<NavLink>` navigate without page reload.
- `useNavigate()` navigates programmatically; `useParams()` reads `/users/:id`; `useSearchParams()` handles query strings.
- Nested routes render children via `<Outlet/>`.
- Protected route: a wrapper that checks auth and returns `<Navigate to='/login' replace />`.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Newer versions (v6.4+/v7) add data APIs: `createBrowserRouter`, loaders, actions, `errorElement`. Mention them to show you are current.

CODE

```
function Protected({ children }) {
  const { user } = useAuth();
  return user ? children : <Navigate to="/login" replace />;
}
<Route path="/dash" element={<Protected><Dash/></Protected>} />
```

Q64. Explain the Redux data flow and its three principles.

ANSWER

Flow: UI dispatches an ACTION -> the STORE passes it (through middleware) to the REDUCER -> reducer returns new STATE -> subscribed components re-render via selectors.

Principles: single source of truth (one store), state is read-only (change only through actions), changes are made with pure reducers.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Middleware (redux-thunk, saga) handles async side effects between dispatch and reducer. Selectors with `useSelector` avoid unnecessary re-renders.

Q65. What is Redux Toolkit (RTK)? Why is it recommended?

ANSWER

RTK is the official, opinionated way to write Redux. `configureStore` sets up the store with good defaults and DevTools; `createSlice` generates actions + reducers together and lets you write 'mutating' logic safely thanks to Immer; `createAsyncThunk` handles async; RTK Query handles data fetching and caching.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

It removes boilerplate and common mistakes from classic Redux.

CODE

```
const counter = createSlice({
  name: 'counter',
  initialState: { value: 0 },
  reducers: { inc: state => { state.value += 1; } },
});
export const { inc } = counter.actions;
```

Q66. Context API vs Redux vs Zustand - which to choose?

ANSWER

- Context: rarely changing global values (theme, auth, locale). No selectors, so frequent updates cause wide re-renders.
- Redux Toolkit: large apps, many developers, complex flows, strong DevTools/middleware, predictable structure.
- Zustand/Jotai: small, minimal-boilerplate stores with selector-based subscriptions - great for mid-size apps.
- TanStack Query/RTK Query: for SERVER state (API data).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Best-practice answer: separate server state (cache with TanStack Query) from client/UI state (useState, Zustand/Redux). Most apps then need very little global client state.

Q67. How do you fetch data in React? How do you avoid race conditions?

ANSWER

Fetch inside `useEffect` (or better, a data library / framework loader). Always handle loading, error and empty states. To prevent race conditions (older slower response overwriting newer one) cancel the request with `AbortController` or use an `ignore` flag in the cleanup.

CODE

```
useEffect(() => {
  const ctrl = new AbortController();
  fetch(`/api/search?q=${q}`, { signal: ctrl.signal })
    .then(r => r.json()).then(setData)
    .catch(e => { if (e.name !== 'AbortError') setErr(e); });
  return () => ctrl.abort();
}, [q]);
```

Q68. Why use TanStack Query (React Query)?**ANSWER**

It manages server state: caching, request de-duplication, background refetching, stale-while-revalidate, retries, pagination/infinite queries, mutations with cache invalidation and optimistic updates - with almost no code.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Using it removes most manual `useEffect` + `useState` fetching code, which is a common source of bugs.

Q69. How do you optimise performance of a React application?**ANSWER**

- Avoid unnecessary re-renders: `React.memo`, `useMemo`, `useCallback`, colocate state, split contexts.
- Code splitting and lazy loading routes/components.
- Virtualise long lists (`react-window` / `TanStack Virtual`).
- Debounce/throttle expensive handlers.
- Use stable keys; avoid inline heavy computation in render.
- Optimise images (lazy loading, WebP, sizes), fonts and bundle size (tree-shaking, analyse with `source-map-explorer`).
- Use transitions (`useTransition`) for heavy updates.
- SSR/SSG, CDN, HTTP caching.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Say: 'first measure with `React DevTools Profiler` and `Lighthouse`, then optimise the bottleneck'. Premature memoisation is an anti-pattern.

Q70. What is list virtualisation / windowing?**ANSWER**

Render only the rows currently visible in the viewport (plus a small buffer) instead of thousands of DOM nodes, by absolute-positioning items inside a tall spacer. Libraries: `react-window`, `react-virtuoso`, `TanStack Virtual`.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Renders 10 000 rows as smoothly as 20.

Q71. How do you implement debounced search in React?**ANSWER**

Keep the raw input in state, derive a debounced value with a `useDebounce` hook (`setTimeout` in effect with `cleanup`), and trigger the API call in an effect that depends on the debounced value.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Implementation is in the Coding Round section. Pair it with `AbortController` to cancel outdated requests.

Q72. Explain the class component lifecycle.**ANSWER**

Mounting: `constructor` -> `render` -> `componentDidMount`. Updating: `render` -> `getSnapshotBeforeUpdate` -> `componentDidUpdate` (triggered by props/state change or `forceUpdate`; `shouldComponentUpdate` can skip). Unmounting: `componentWillUnmount`. Error: `getDerivedStateFromError`, `componentDidCatch`.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Even if you use hooks, freshers are often asked this. Deprecated ones: `componentWillMount`, `componentWillReceiveProps`, `componentWillUpdate` (UNSAFE_).

Q73. When does React preserve or reset component state?**ANSWER**

State is tied to a component's position in the tree and its type. Same component type at the same position keeps state across renders. Different type, or a different `key`, resets it.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Trick: change `key` to intentionally reset state, e.g. `<Profile key={userId} />` so the form clears when the user changes. Defining a component inside another component's body creates a new type each render and destroys its state - never do that.

Q74. How do you test React components?**ANSWER**

Jest (or Vitest) as test runner + React Testing Library (RTL) to render components and query the DOM the way users do (`getByRole`, `getByText`), `user-event` to simulate interactions, MSW to mock network. Cypress/Playwright for end-to-end tests.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Test behaviour, not implementation details (do not test state variables or internal methods). Query priority: `getByRole` > `getByLabelText` > `getByText` > `getById`.

Q75. What are the different ways to style React components?**ANSWER**

Plain CSS / CSS Modules (scoped class names), Sass, CSS-in-JS (styled-components, Emotion - runtime cost, less favoured with Server Components), utility-first Tailwind CSS, component libraries (MUI, Chakra, shadcn/ui, Ant Design), inline styles (no pseudo-classes/media queries).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Mention trade-offs: Tailwind/CSS Modules are zero-runtime and work well with RSC.

Q76. How do you type React with TypeScript (props, events, hooks)?**ANSWER**

Define an interface/type for props. Use `useState<User | null>(null)` for generics, `React.ChangeEvent<HTMLInputElement>` for events, `ReactNode` for children, `React.FC` is optional and not required.

CODE

```
interface Props { title: string; children?: React.ReactNode }
const [user, setUser] = useState<User | null>(null);
const onChange = (e: React.ChangeEvent<HTMLInputElement>) =>
  setText(e.target.value);
```

Q77. How do you implement authentication in a React app?**ANSWER**

Login form -> API returns token(s). Prefer HttpOnly, Secure, SameSite cookies for tokens (safe from XSS). Keep the user in an AuthContext/store, protect routes with a guard component, attach credentials/token in an axios/fetch interceptor, refresh tokens silently, and clear state on logout/401.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Avoid storing JWTs in localStorage for sensitive apps (XSS can steal them). Client-side route guards are only UX - the server must enforce authorisation.

Q78. What is lazy state initialisation?

ANSWER

Pass a function to `useState` so the expensive initial computation runs only on the first render: `useState(() => JSON.parse(localStorage.getItem('k')))`. Writing `useState(expensive())` runs `expensive()` on every render (result is ignored after the first).

Q79. What is the difference between server state and client state?

ANSWER

Server state is remote data you do not own (users, products): it is async, can become stale and is shared. Client state is local UI data (modal open, form input, theme) that you fully own and is synchronous.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Managing them with different tools (TanStack Query vs `useState/Zustand`) is the modern best practice.

Q80. What is `useId` and why was it introduced?

ANSWER

`useId()` generates a unique, stable ID that matches between server and client, used to link `label` `htmlFor` and `input` `id` or ARIA attributes.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Do not use it for list keys. Avoids hydration mismatches caused by `Math.random()` or counters.

Level 3 - Advanced React (3+ Years / Senior)

Internals, concurrent rendering, SSR/Server Components, React 19, architecture, security and scalability.

Q81. Explain React Fiber architecture.

ANSWER

Fiber is the reconciliation engine introduced in React 16. Each element becomes a 'fiber' - a JS object holding type, props, state, and pointers (child, sibling, return) plus effect info. Rendering work is broken into small units that can be paused, resumed, prioritised or aborted, instead of one uninterruptible recursive pass.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Fiber enables concurrent features: time slicing, Suspense, transitions. React keeps two trees: the current tree (on screen) and the work-in-progress tree; after commit they swap.

Q82. Explain the render phase vs commit phase.

ANSWER

Render phase (pure, can be paused/restarted): React calls your components and diffs the trees to compute what changed. Commit phase (synchronous, cannot be interrupted): React applies changes to the DOM, then runs layout effects, and later passive effects (useEffect).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Because render may run multiple times or be discarded, render must be pure. Side effects belong only in the commit phase (effects and handlers).

Q83. What is concurrent rendering? Explain useTransition and useDeferredValue.

ANSWER

Concurrent rendering lets React prepare multiple versions of the UI and interrupt low-priority work to handle urgent updates (typing, clicks).

- `useTransition` returns `[isPending, startTransition]`; updates inside `startTransition` are marked non-urgent and can be interrupted.
- `useDeferredValue(value)` returns a lagging copy of a value that updates after urgent work is done.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Use for expensive filtering or tab switching while keeping the input responsive. It is not debouncing - there is no fixed delay and work can be interrupted.

CODE

```
const [isPending, startTransition] = useTransition();
function onChange(e) {
  setQuery(e.target.value); // urgent
  startTransition(() => setFiltered(filter(e.target.value)));
}
```

Q84. What is automatic batching in React 18?

ANSWER

React 18 batches state updates everywhere (event handlers, promises, `setTimeout`, native events) into one re-render when you use `createRoot`. React 17 batched only inside React event handlers.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Opt out with `flushSync` when you need DOM updated immediately.

Q85. Explain Suspense in depth. What is streaming SSR?

ANSWER

Suspense lets a component 'wait' for something (lazy code, data via `use()` or a Suspense-enabled library) and shows a fallback meanwhile, isolating loading states to boundaries. Streaming SSR sends HTML in chunks: the shell is sent immediately, suspended parts stream in later and are hydrated selectively (selective hydration).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Benefits: faster Time-to-First-Byte/First Contentful Paint, no all-or-nothing hydration, and parts the user interacts with are prioritised.

Q86. CSR vs SSR vs SSG vs ISR? What is hydration?

ANSWER

- CSR: browser gets empty HTML, JS renders everything (slow first paint, poor SEO out of the box).
- SSR: server renders HTML per request (good SEO/first paint, needs a server).
- SSG: HTML generated at build time, served from CDN (fastest, static data).
- ISR: static pages regenerated in background after an interval or on demand.

Hydration: React attaches event handlers and state to server-rendered HTML in the browser without recreating the DOM.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Choose by data freshness and SEO needs: marketing pages -> SSG, product pages -> ISR, dashboards -> CSR, personalised pages -> SSR.

Q87. What is a hydration mismatch and how do you fix it?

ANSWER

It occurs when server-rendered HTML differs from what the client renders on first render (e.g. `Date.now()`, `Math.random()`, `window.innerWidth`, locale/timezone differences, invalid HTML nesting, browser extensions). React warns and may discard server HTML.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Fixes: render deterministic output on first pass, move browser-only values into `useEffect` (client-only after mount), use `useId`, use `suppressHydrationWarning` for unavoidable text differences.

Q88. What are React Server Components (RSC)?

ANSWER

Components that run only on the server (during build or request) and send their rendered result (RSC payload) to the client, with zero JS shipped for them. They can directly access databases, file system and secrets, and use `async/await`. Client Components (marked with `'use client'`) run in the browser and hold state, effects and event handlers.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Server components cannot use `useState`, `useEffect` or browser APIs; client components cannot import server components (but can receive them as children). Benefits: smaller bundles, faster loads, secure data access. Popular in Next.js App Router.

Q89. What are Server Actions / 'use server'?

ANSWER

Async functions marked with `'use server'` that run on the server but can be called directly from client components or bound to `<form action={fn}>`. React handles the network round trip.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

They replace hand-written API routes for mutations. Always validate input and check authorisation inside them - they are public HTTP endpoints.

Q90. What is new in React 19?

ANSWER

- Actions: async functions in transitions; `useActionState` (form state + pending + result), `useFormStatus`, `useOptimistic` for instant UI.
- `use()` API reads promises and context (even conditionally).
- `ref` as a normal prop; ref callbacks can return cleanup.
- Context as provider: `<Ctx value={...}>` instead of `<Ctx.Provider>`.
- Native document metadata tags (`<title>`, `<meta>`) hoisted to head.
- Stable Server Components and Server Actions.
- Better error reporting and hydration diffs; support for Web Components.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Being able to name 3-4 of these and explain `useOptimistic/useActionState` will impress an interviewer.

CODE

```
const [state, formAction, isPending] = useActionState(
  async (prev, formData) => save(formData), null);
<form action={formAction}>
  <input name="email" />
  <button disabled={isPending}>Save</button>
</form>
```

Q91. What is the React Compiler?

ANSWER

A build-time compiler (formerly 'React Forget') that automatically memoises components and hooks by analysing code, so you rarely need to write `useMemo`, `useCallback` or `React.memo` manually. It requires code that follows the Rules of React (purity, no mutation).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Interview tip: knowing manual memoisation is still necessary knowledge because the compiler is opt-in/newer and many codebases predate it.

Q92. What is `useSyncExternalStore`?

ANSWER

A hook for subscribing to external stores (Redux, Zustand, browser APIs like `navigator.onLine`) in a way that is safe with concurrent rendering (prevents tearing - different parts of UI showing different snapshots).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

You give it `subscribe`, `getSnapshot` and optional `getServerSnapshot`. Libraries use it internally; you rarely call it directly.

Q93. What is `useImperativeHandle`?

ANSWER

It customises the instance value exposed to a parent via ref, so the parent can call only chosen methods (e.g. `focus()`, `reset()`) instead of getting the raw DOM node.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Use sparingly; prefer props/state for communication. Imperative APIs are ok for focus, scroll, media playback.

CODE

```
useImperativeHandle(ref, () => ({
  focus: () => inputRef.current.focus(),
}), []);
```

Q94. 'You might not need an effect' - explain.**ANSWER**

Many effects are unnecessary. Don't use effects to: (1) derive state from props/state - compute during render; (2) reset state when a prop changes - use `key`; (3) handle user events - use event handlers; (4) chain state updates; (5) notify parent - call callback in the event handler; (6) fetch data if a data library/framework can do it.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Effects are only for syncing with external systems (network, DOM APIs, subscriptions, analytics). Fewer effects = fewer bugs.

Q95. How does React's event system work internally?**ANSWER**

React attaches one listener per event type on the root container (event delegation) and dispatches to handlers using its own tree. Events are wrapped in SyntheticEvent objects. Since React 17 the root container is used (not document) allowing multiple React versions to coexist; event pooling was removed so events can be used asynchronously.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Order: React capture -> target -> bubble. `e.stopPropagation()` stops React propagation; use `e.nativeEvent.stopImmediatePropagation()` if mixing with native listeners.

Q96. How would you decide on a state-management architecture for a large app?**ANSWER**

1. Keep state as local as possible (useState).
2. Server data -> TanStack Query/RTK Query.
3. URL state (filters, pagination) -> search params.
4. Form state -> React Hook Form.
5. Shared UI/global client state -> Zustand/Redux Toolkit/Context depending on update frequency and team size.
6. Derived data -> compute with selectors/useMemo, do not store.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Show you think about categories of state, not just tools.

Q97. How do you avoid unnecessary re-renders caused by Context?**ANSWER**

Split contexts (state and dispatch separate, or per domain), memoise the provider value, move state down so only relevant subtree re-renders, wrap consumers in memo with selectors via `use-context-selector`, or use an external store with selectors (Zustand/Redux).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Passing `children` through a provider component prevents children from re-rendering when provider's own state changes, because the children elements have the same reference.

Q98. What are the pitfalls of over-memoisation?**ANSWER**

Memoisation has costs: memory, comparison work every render, and more complex code. Wrong dependency arrays cause stale values. Memoising cheap components gives no gain. Unstable props defeat it anyway.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Rule: profile first; memoise expensive calculations and components that re-render often with same props; otherwise keep it simple.

Q99. How do you secure a React application?

ANSWER

- XSS: React escapes JSX output by default; avoid `dangerouslySetInnerHTML` (sanitise with DOMPurify), validate URLs (`javascript:` in href), never `eval`.
- Tokens: HttpOnly Secure SameSite cookies; short-lived access tokens.
- CSRF: SameSite cookies/CSRF tokens for cookie auth.
- Dependencies: audit (`npm audit`, Dependabot), lockfile, avoid untrusted packages.
- CSP headers, HTTPS, CORS configured on server.
- Do not expose secrets in client env vars (anything in the bundle is public).
- Authorisation always on the server.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Frontend cannot hide secrets; anything shipped to the browser is visible.

Q100. How do you make a React app accessible (a11y)?

ANSWER

Use semantic HTML (`button`, `nav`, `main`), label inputs, keyboard navigation and visible focus, manage focus in modals/route changes, ARIA only when native semantics are insufficient, sufficient colour contrast, alt text, `aria-live` for dynamic updates, test with `eslint-plugin-jsx-a11y`, axe, screen readers.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

A `div onClick` is not a button - it lacks keyboard and role support.

Q101. What are Core Web Vitals and how do you improve them in React?

ANSWER

LCP (Largest Contentful Paint, loading), INP (Interaction to Next Paint, responsiveness - replaced FID), CLS (Cumulative Layout Shift, visual stability).

Improve: SSR/SSG and preloading hero image (LCP); code splitting, transitions, avoid long tasks, memoisation (INP); set width/height for images and reserve space for async content (CLS).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Measure with Lighthouse, PageSpeed Insights, web-vitals library, Chrome Performance tab and React Profiler.

Q102. How do you structure a large React project?

ANSWER

Feature-based (domain) structure scales best:

`src/features/auth/{components, hooks, api, slice, types}`, `src/components/ui` (shared design system), `src/hooks`, `src/lib` (axios, utils), `src/routes`, `src/store`.

Rules: colocate code by feature, public API through index files, avoid cross-feature imports, absolute imports, consistent naming, barrel files carefully (can hurt tree-shaking).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Mention monorepos (Turborepo/Nx) for multiple apps sharing a design system.

Q103. What are micro-frontends? Module Federation?

ANSWER

An architecture where a big frontend is split into independently developed and deployed apps composed at runtime (iframes, Web Components, or Webpack/Vite Module Federation sharing modules and dependencies dynamically).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Pros: independent deployments, team autonomy. Cons: complexity, duplicate dependencies, inconsistent UX, performance overhead. Use only for large orgs.

Q104. What is your testing strategy for a React app?

ANSWER

Testing pyramid: many unit tests (utils, hooks, reducers) -> integration tests with RTL for components/pages with mocked API (MSW) -> a few E2E tests (Playwright/Cypress) for critical flows (login, checkout). Add type checking, lint, visual regression (Storybook/Chromatic) and CI gates.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Focus on user-visible behaviour and avoid snapshot spam.

Q105. Design an autocomplete / search-as-you-type component (frontend system design).

ANSWER

Requirements: debounced input, cancel stale requests, keyboard navigation (up/down/enter/esc), highlight matches, loading/empty/error states, ARIA combobox roles, cache recent queries, limit results, handle click outside.

Approach: controlled input + `useDebounce` + fetch with AbortController + in-memory cache (Map) + `activeIndex` state + `role='combobox'/listbox/option`.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Interviewers evaluate structure of thinking: requirements, data flow, edge cases, performance, accessibility.

Q106. Design an infinite scrolling feed.

ANSWER

Use IntersectionObserver on a sentinel element to load the next page (cursor-based pagination), TanStack `useInfiniteQuery` for caching, virtualise items to keep DOM small, handle loading/error/end states, preserve scroll position, de-duplicate items, and provide a 'retry' control.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Cursor pagination beats offset pagination for feeds that change frequently.

Q107. What is Immer and why is it used with Redux Toolkit?

ANSWER

Immer lets you write code that appears to mutate a draft state; it tracks changes and produces a new immutable state via structural sharing. RTK's `createSlice` uses it internally.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

It makes deeply nested updates readable while guaranteeing immutability.

Q108. Explain reconciliation of keyed children in detail. What happens when you insert an item at the start of a list with index keys?

ANSWER

React matches children by key (then by type/position). With index keys, inserting at the start shifts every item's index, so React thinks item 0 became a different item: it updates every row's props, and any internal state or uncontrolled input values remain at the old index. With stable id keys React inserts one new node and moves nothing.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Explains both performance and correctness problems of index keys.

Q109. How does useEffect timing work? What is the order of effects between parent and child?

ANSWER

On mount: render parent -> render child, then effects run child first, then parent (children's effects and DOM mutations complete before parents'). Cleanups run before the effect re-runs and on unmount; never depend on the relative order of cleanups between components.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Use the mental model: 'effects are independent synchronisation processes' and do not rely on cross-effect ordering.

Q110. How do you implement optimistic updates?

ANSWER

Immediately update the UI as if the request succeeded (`useOptimistic` in React 19, or TanStack Query `onMutate` updating cache), send the request, and roll back (`onError`) or replace with server truth (`onSettled/invalidate`) if it fails.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Perfect for like buttons, todo toggles, chat messages.

Q111. How do you handle errors globally in a React application?

ANSWER

Error Boundaries for render errors (route-level + app-level fallback), `window.onerror` and `unhandledrejection` listeners for uncaught errors, centralised axios/fetch interceptors for API errors (toast, redirect on 401), TanStack Query global `onError`, logging to Sentry/LogRocket with user context and release tags.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Distinguish expected errors (validation) from unexpected ones (bugs).

Q112. Next.js vs plain React (Vite)? What does Next.js add?

ANSWER

Next.js is a full-stack React framework providing file-based routing, SSR/SSG/ISR, React Server Components (App Router), API routes/route handlers, image/font optimisation, middleware, caching and bundling defaults. Plain React + Vite is a client-side SPA needing you to add routing and SSR yourself.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Choose Next.js when SEO, performance or full-stack features matter; Vite SPA for internal dashboards or apps behind login.

Q113. How would you migrate a class-component codebase to hooks?

ANSWER

Migrate incrementally - hooks and classes can coexist. Start with leaf components, extract shared logic into custom hooks, replace HOCs/render props with hooks, add tests first, keep Error Boundaries as classes, and use codemods/lint rules to guard consistency.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Show a risk-managed, gradual approach rather than a rewrite.

Q114. What is the difference between useMemo caching and React's own rendering optimisation like bailouts?

ANSWER

If setState is called with the same value (by Object.is), React bails out and skips re-rendering children (it may render the component once more before bailing). Bailout also happens when a component returns the same element reference. useMemo is a developer-controlled cache for values; bailout is automatic.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Do not rely on useMemo for semantic guarantees - React may discard cache in the future.

Level 4 - Tricky and Output-Based Questions

Interviewers love 'what will this print / render?' questions because they test whether you truly understand how React works. Read the code, predict the answer, then check.

Q115. What is the final count after one click?

ANSWER

The count increases by 1, not 3.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

All three calls read the same `count` value from the current render (0), so each schedules `setCount(0 + 1)`. React batches them and the last one wins. Use the functional form `setCount(c => c + 1)` three times to get 3.

CODE

```
function App() {
  const [count, setCount] = useState(0);
  const add = () => {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
  };
  return <button onClick={add}>{count}</button>;
}
```

Q116. What does `console.log` print right after `setState`?

ANSWER

It prints the OLD value.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

State is a snapshot per render. The setter schedules the next render; the current closure still holds the old value. To act on the new value, use an effect that depends on it, or compute the next value in a variable first.

CODE

```
const [n, setN] = useState(0);
function click() {
  setN(n + 1);
  console.log(n); // 0, not 1
}
```

Q117. Why is the counter stuck at 1 in this component?

ANSWER

The interval callback closes over `count` from the first render (0), so it always sets 1.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Effect with `[]` captures the initial render's variables. Fix with `setCount(c => c + 1)` or include `count` in deps (which recreates the interval each tick).

CODE

```
useEffect(() => {
  const id = setInterval(() => setCount(count + 1), 1000);
  return () => clearInterval(id);
}, []);
```

Q118. Why does the input text jump to the wrong row after deleting an item?

ANSWER

Because index is used as key.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

After deleting row 0, the old row 1 becomes index 0 and reuses the DOM node/state of the old row 0. Use a unique stable id as key.

CODE

```
{items.map((item, i) => (
  <li key={i}><input defaultValue={item.name} /></li>
))}
```

Q119. What is the order of console output when this component mounts?**ANSWER**

Output order: Parent render, Child render, Child effect, Parent effect.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Render goes top-down (parent then child). Effects run bottom-up (child effects before parent effects) after the commit.

CODE

```
function Parent() {
  console.log('Parent render');
  useEffect(() => console.log('Parent effect'), []);
  return <Child />;
}
function Child() {
  console.log('Child render');
  useEffect(() => console.log('Child effect'), []);
  return null;
}
```

Q120. Does this effect run on every render? Why?**ANSWER**

Yes, effectively it runs every render.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

`{}` and `[]` literals create new references each render, so `Object.is` comparison always says 'changed'. Memoise, or depend on primitives.

CODE

```
const filters = { status: 'active' };
useEffect(() => { load(filters); }, [filters]);
```

Q121. What does this render when the list is empty?**ANSWER**

It renders the number 0 on screen.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

`0 && <X/>` evaluates to 0, and React renders numbers. Use `list.length > 0 && ...` or a ternary.

CODE

```
{list.length && <List data={list} />}
```

Q122. What is wrong with this button? (onClick={handle()})**ANSWER**

`handle()` is invoked during render, and its return value (undefined) is assigned as the handler. It may also cause an infinite render loop if it sets state.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Pass a function reference: `onClick={handle}` or `onClick={() => handle(id)}`.

CODE

```
<button onClick={handle()}>Save</button>
```

Q123. Why doesn't the UI update here?**ANSWER**

The array was mutated and the same reference was passed to setState.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

React sees the same reference and bails out. Create a new array.

CODE

```
const [list, setList] = useState([1, 2]);
const add = () => { list.push(3); setList(list); };
// FIX: setList([...list, 3]);
```

Q124. What error will this code throw?**ANSWER**

'Rendered more hooks than during the previous render' (or hook order changed).

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Hooks must be called in the same order every render. Move the condition inside the hook or split into components.

CODE

```
function Profile({ user }) {
  if (!user) return null;
  const [tab, setTab] = useState('a'); // conditional hook
  ...
}
```

Q125. Will the child re-render when the parent re-renders? (React.memo used)**ANSWER**

Yes, it will still re-render.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

The inline arrow function is a new reference each render, so memo's shallow prop comparison fails. Wrap it with useCallback.

CODE

```
const Child = React.memo(({ onClick }) => <button onClick={onClick}/>);
function Parent() {
  const [n, setN] = useState(0);
  return <Child onClick={() => console.log('hi')} />;
}
```

Q126. Why does React warn about this useEffect?**ANSWER**

An effect callback must return either nothing or a cleanup function; an async function returns a Promise.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Define an async function inside the effect and call it.

CODE

```
useEffect(async () => { // wrong
  const res = await fetch(url);
}, []);
```

Q127. What happens when setState is called with the same value?**ANSWER**

React bails out: it may render that component once more but will not re-render children or run effects.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

It compares using Object.is. `setCount(5)` when count is already 5 is (almost) free. For objects, a new object with the same content is still a different reference and will re-render.

Q128. What does `useState(expensive())` vs `useState(expensive)` do?**ANSWER**

`useState(expensive())` calls `expensive` on every render (only the first result is used). `useState(expensive)` (function reference, or `() => expensive()`) calls it only on the first render.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Lazy initialisation is important for parsing `localStorage`, heavy calculations.

Q129. Predict the output of this classic event-loop + closure question.**ANSWER**

Prints 3 3 3 with `var`; 0 1 2 with `let`.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

`var` is function-scoped so all callbacks share one `i` (which is 3 when they run). `let` creates a new binding per iteration.

CODE

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}
```

Q130. Why does the modal component below lose its input text on every keystroke?**ANSWER**

Because a component is defined inside another component, creating a new component type every render.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

React sees a different type at the same position, unmounts the old subtree (state lost) and mounts a new one. Always define components at module level.

CODE

```
function Page() {  
  const [t, setT] = useState('');  
  const Form = () => <input value={t} onChange={e => setT(e.target.value)} />;  
  return <Form />;  
}
```

Level 5 - Machine Coding / Live Coding Round

Most product-company rounds ask you to build a small component or hook in 30-60 minutes. Practise each of these until you can write them from memory. Talk through requirements, edge cases and accessibility while coding.

Q131. Implement a `useDebounce` hook.

ANSWER

Return a value that updates only after the input value has stopped changing for `delay` ms.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

The cleanup clears the previous timer each time value changes, so only the last change fires.

CODE

```
function useDebounce(value, delay = 500) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}
```

Q132. Implement `useLocalStorage`.

ANSWER

A hook that behaves like `useState` but persists to `localStorage`.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Uses lazy initialisation and try/catch (storage may be blocked). Sync in an effect.

CODE

```
function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    try {
      const raw = localStorage.getItem(key);
      return raw ? JSON.parse(raw) : initial;
    } catch { return initial; }
  });
  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);
  return [value, setValue];
}
```

Q133. Implement `usePrevious`.

ANSWER

Store the value from the previous render using a ref updated in an effect.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

The effect runs after render, so during render `ref.current` still holds the previous value.

CODE

```
function usePrevious(value) {
  const ref = useRef();
  useEffect(() => { ref.current = value; }, [value]);
  return ref.current;
}
```

Q134. Implement a `useFetch` hook with loading, error and cancellation.**ANSWER**

Track data/loading/error state and abort the request on cleanup or when url changes.

CODE

```
function useFetch(url) {
  const [state, setState] = useState(
    { data: null, loading: true, error: null });
  useEffect(() => {
    const ctrl = new AbortController();
    setState({ data: null, loading: true, error: null });
    fetch(url, { signal: ctrl.signal })
      .then(r => { if (!r.ok) throw new Error(r.status); return r.json(); })
      .then(data => setState({ data, loading: false, error: null }))
      .catch(error => {
        if (error.name !== 'AbortError')
          setState({ data: null, loading: false, error });
      });
    return () => ctrl.abort();
  }, [url]);
  return state;
}
```

Q135. Build a Todo app (add, toggle, delete, filter).**ANSWER**

Use array state with unique ids; immutably update with map/filter; derive the filtered list during render instead of storing it.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Discuss: keys, controlled input, empty state, persistence (localStorage), useReducer alternative, accessibility (label, button types).

CODE

```
function Todos() {
  const [todos, setTodos] = useState([]);
  const [text, setText] = useState('');
  const [filter, setFilter] = useState('all');
  const add = e => {
    e.preventDefault();
    if (!text.trim()) return;
    setTodos(t => [...t, { id: crypto.randomUUID(), text, done: false }]);
    setText('');
  };
  const toggle = id => setTodos(t =>
    t.map(x => x.id === id ? { ...x, done: !x.done } : x));
  const remove = id => setTodos(t => t.filter(x => x.id !== id));
  const shown = todos.filter(t =>
    filter === 'all' ? true : filter === 'done' ? t.done : !t.done);
  return (
    <>
      <form onSubmit={add}>
        <input value={text} onChange={e => setText(e.target.value)} />
        <button>Add</button>
      </form>
      {shown.map(t => (
        <li key={t.id}>
          <input type="checkbox" checked={t.done}
            onChange={() => toggle(t.id)} />
          {t.text}
          <button onClick={() => remove(t.id)}>x</button>
        </li>
      ))}
    </>
  );
}
```

Q136. Build an Accordion (only one panel open at a time).**ANSWER**

Keep `openIndex` in the parent; each item toggles it. Use button elements and `aria-expanded`.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Extension questions: allow multiple open (use a Set), animate height, keyboard support.

CODE

```
function Accordion({ items }) {
  const [open, setOpen] = useState(null);
  return items.map((it, i) => (
    <div key={it.id}>
      <button aria-expanded={open === i}
        onClick={() => setOpen(open === i ? null : i)}>
        {it.title}
      </button>
      {open === i && <p>{it.body}</p>}
    </div>
  ));
}
```

Q137. Build a Tabs component.**ANSWER**

Store the active tab id in state; render tab buttons and only the active panel.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Add `role='tablist'`, `role='tab'`, `aria-selected`, and arrow-key navigation for bonus points.

CODE

```
function Tabs({ tabs }) {
  const [active, setActive] = useState(tabs[0].id);
  return (
    <div>
      <div role="tablist">
        {tabs.map(t => (
          <button key={t.id} role="tab"
            aria-selected={active === t.id}
            onClick={() => setActive(t.id)}>{t.label}</button>
        ))}
      </div>
      <div role="tabpanel">
        {tabs.find(t => t.id === active).content}
      </div>
    </div>
  );
}
```

Q138. Build a Modal using a Portal.**ANSWER**

Render into `document.body` with `createPortal`, close on Escape and overlay click, lock scroll, and restore focus on close.

CODE

```
function Modal({ open, onClose, children }) {
  useEffect(() => {
    const onKey = e => e.key === 'Escape' && onClose();
    document.addEventListener('keydown', onKey);
    return () => document.removeEventListener('keydown', onKey);
  }, [onClose]);
  if (!open) return null;
  return createPortal(
    <div className="overlay" onClick={onClose}>
      <div role="dialog" aria-modal="true"
        onClick={e => e.stopPropagation()}>
        {children}
      </div>
    </div>, document.body);
}
```

Q139. Build a Star Rating component.

ANSWER

Keep **rating** and **hover** in state; render N stars where a star is filled if index < (hover || rating).

CODE

```
function Stars({ max = 5 }) {
  const [rating, setRating] = useState(0);
  const [hover, setHover] = useState(0);
  return [...Array(max)].map((_, i) => {
    const n = i + 1;
    return (
      <span key={n} style={{ cursor: 'pointer',
        color: n <= (hover || rating) ? 'gold' : '#ccc' }}
        onClick={() => setRating(n)}
        onMouseEnter={() => setHover(n)}
        onMouseLeave={() => setHover(0)}>&#9733;</span>
    );
  });
}
```

Q140. Implement infinite scroll with IntersectionObserver.

ANSWER

Render a sentinel div at the bottom; when it becomes visible, load the next page. Guard against duplicate loads and stop at the end.

CODE

```
function Feed() {
  const [items, setItems] = useState([]);
  const [page, setPage] = useState(1);
  const sentinel = useRef(null);
  useEffect(() => {
    fetch(`/api/items?page=${page}`)
      .then(r => r.json())
      .then(d => setItems(prev => [...prev, ...d]));
  }, [page]);
  useEffect(() => {
    const io = new IntersectionObserver([e] => {
      if (e.isIntersecting) setPage(p => p + 1);
    });
    io.observe(sentinel.current);
    return () => io.disconnect();
  }, []);
  return <>{items.map(i => <div key={i.id}>{i.title}</div>)}
    <div ref={sentinel} /></>
</>
}
```

Q141. Build a debounced search / autocomplete.

ANSWER

Combine `useDebounce` with an effect that fetches when the debounced query changes and aborts stale requests.

CODE

```
function Search() {
  const [q, setQ] = useState('');
  const dq = useDebounce(q, 400);
  const [res, setRes] = useState([]);
  useEffect(() => {
    if (!dq) return setRes([]);
    const c = new AbortController();
    fetch(`/api/search?q=${dq}`, { signal: c.signal })
      .then(r => r.json()).then(setRes).catch(() => {});
    return () => c.abort();
  }, [dq]);
  return <>
    <input value={q} onChange={e => setQ(e.target.value)} />
    <ul>{res.map(r => <li key={r.id}>{r.name}</li>)}</ul>
  </>;
}
```

Q142. Build a Pagination component.

ANSWER

State: `page`. Derive `totalPages = Math.ceil(total / pageSize)` and slice data. Disable Prev/Next at boundaries.

CODE

```
function Paginated({ data, pageSize = 10 }) {
  const [page, setPage] = useState(1);
  const total = Math.ceil(data.length / pageSize);
  const slice = data.slice((page - 1) * pageSize, page * pageSize);
  return <>
    {slice.map(d => <p key={d.id}>{d.name}</p>)}
    <button disabled={page === 1} onClick={() => setPage(p => p - 1)}>Prev</button>
    <span>{page} / {total}</span>
    <button disabled={page === total} onClick={() => setPage(p => p + 1)}>Next</button>
  </>;
}
```

Q143. Build an OTP input (4 boxes with auto-focus).

ANSWER

Keep an array of digits and an array of refs; on change move focus to the next box; on Backspace move to the previous.

CODE

```
function Otp({ length = 4 }) {
  const [vals, setVals] = useState(Array(length).fill(''));
  const refs = useRef([]);
  const change = (i, v) => {
    if (!/^^\d?$/.test(v)) return;
    const next = [...vals]; next[i] = v; setVals(next);
    if (v && i < length - 1) refs.current[i + 1].focus();
  };
  const key = (i, e) => {
    if (e.key === 'Backspace' && !vals[i] && i > 0)
      refs.current[i - 1].focus();
  };
  return vals.map((v, i) => (
    <input key={i} maxLength={1} value={v}
      ref={el => (refs.current[i] = el)}
      onChange={e => change(i, e.target.value)}
      onKeyDown={e => key(i, e)} />
  ));
}
```

Q144. Implement `useOnClickOutside`.

ANSWER

Attach a document listener that calls the handler if the event target is not inside the ref element.

CODE

```
function useOnClickOutside(ref, handler) {
  useEffect(() => {
    const listener = e => {
      if (!ref.current || ref.current.contains(e.target)) return;
      handler(e);
    };
    document.addEventListener('mousedown', listener);
    return () => document.removeEventListener('mousedown', listener);
  }, [ref, handler]);
}
```

Q145. Build a countdown timer / stopwatch.

ANSWER

Keep **seconds** and **running** in state; run `setInterval` in an effect only while running, with cleanup. Use the functional updater.

CODE

```
function Timer() {
  const [s, setS] = useState(0);
  const [run, setRun] = useState(false);
  useEffect(() => {
    if (!run) return;
    const id = setInterval(() => setS(x => x + 1), 1000);
    return () => clearInterval(id);
  }, [run]);
  return <>
    <h2>{s}</h2>
    <button onClick={() => setRun(r => !r)}>{run ? 'Pause' : 'Start'}</button>
    <button onClick={() => { setRun(false); setS(0); }}>Reset</button>
  </>;
}
```

Q146. Build a recursive file explorer / nested comments.

ANSWER

A component that renders itself for children; each node keeps its own **open** state.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Recursive components are common; discuss key usage and performance for big trees.

CODE

```
function Node({ node }) {
  const [open, setOpen] = useState(false);
  return (
    <div style={{ marginLeft: 16 }}>
      <span onClick={() => setOpen(o => !o)}>
        {node.children ? (open ? '- ' : '+ ') : ''}{node.name}
      </span>
      {open && node.children?.map(c => <Node key={c.id} node={c} />)}
    </div>
  );
}
```

Q147. Write debounce and throttle in plain JavaScript.

ANSWER

Debounce keeps one timer and resets it on each call. Throttle runs the function immediately and ignores calls until the wait period passes.

CODE

```
function debounce(fn, wait) {
  let t;
  return function (...args) {
    clearTimeout(t);
    t = setTimeout(() => fn.apply(this, args), wait);
  };
}

function throttle(fn, wait) {
  let last = 0;
  return function (...args) {
    const now = Date.now();
    if (now - last >= wait) { last = now; fn.apply(this, args); }
  };
}
```

Q148. Build a Counter with useReducer and a custom hook useCounter.

ANSWER

Encapsulate state and actions in a custom hook returning `{count, inc, dec, reset}`.

CODE

```
function useCounter(initial = 0) {
  const [count, dispatch] = useReducer((s, a) => {
    if (a === 'inc') return s + 1;
    if (a === 'dec') return s - 1;
    return initial;
  }, initial);
  return { count,
    inc: () => dispatch('inc'), dec: () => dispatch('dec'),
    reset: () => dispatch('reset') };
}
```

Level 6 - Behavioural, Project and HR Questions

Technical skill gets you shortlisted; clear communication gets you the offer. Prepare structured answers (situation, action, result) using your own projects.

Q149. Tell me about yourself.

ANSWER

Use a 60-90 second structure: who you are (degree/year) -> what you built or learned in React (2 projects, key tech) -> one achievement -> why this role/company.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Example: 'I am a B.Tech CSE student who builds frontend apps with React. I built X (auth, REST API, Redux Toolkit) and deployed it on Vercel. I recently improved its Lighthouse score from 60 to 92 by code splitting and image optimisation. I am looking for a frontend role where I can grow on production-grade React.' Replace with your real facts.

Q150. Explain your best React project end to end.

ANSWER

Cover: problem it solves, tech stack and why, architecture (folder structure, state management choice, routing, API layer), the hardest bug or challenge and how you fixed it, performance/accessibility work, deployment, and what you would improve.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Be ready to open the code. Interviewers pick any file and ask 'why did you write it this way?'. Know every dependency in your package.json.

Q151. What was the biggest challenge you faced and how did you solve it?

ANSWER

Pick a concrete technical problem (stale state bug, slow list, CORS, auth token refresh). Describe symptom, how you debugged (DevTools, Profiler, logs, minimal repro), root cause, fix, and lesson learned.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Shows problem-solving process, which matters more than the specific bug.

Q152. How do you debug a React application?

ANSWER

React DevTools (components tree, props/state, Profiler flamegraph), browser DevTools (Network, Console, breakpoints, Performance), console.log with labels, StrictMode warnings, ESLint hooks rules, error boundaries/Sentry logs, reproduce in a minimal sandbox, binary-search the recent changes with git bisect.

Q153. Why did you choose React for your project?

ANSWER

Component reusability, declarative UI, huge ecosystem/community, strong job market, ease of integrating with any backend, and knowledge that transfers to React Native.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Show you also understand when React is not ideal (e.g. static content site could use Astro).

Q154. How do you keep yourself updated with React?**ANSWER**

Official docs (react.dev), React blog/changelog, Kent C. Dodds / Dan Abramov posts, Josh Comeau, newsletters (React Status, Bytes), YouTube, building side projects with new APIs, and reading source of libraries you use.

Q155. You are a fresher with no work experience - why should we hire you?**ANSWER**

Highlight strong fundamentals, real projects deployed to production-like environments, GitHub activity, certifications, ability to learn fast (give example), and willingness to take feedback.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Back claims with evidence: live links, commits, a code review you responded to, open-source contribution.

Q156. How do you handle code reviews and disagreements in a team?**ANSWER**

Treat review as learning: ask clarifying questions, separate opinion from principle, back decisions with data or docs, be open to change, agree on team conventions/lint rules, and keep feedback about code, not people.

Q157. What questions should you ask the interviewer?**ANSWER**

Ask about: the tech stack and React version, how they manage state/testing, onboarding and mentorship, what a successful first 90 days looks like, code review culture, product roadmap, and team structure.

EXPLANATION / WHAT INTERVIEWERS LOOK FOR

Good questions show genuine interest and maturity.

Rapid-Fire Questions (Answer in One Line)

Quick fire questions that appear in first screening rounds. Practise answering each in one or two sentences.

Q158. Rapid-fire set 1

ANSWER

- Is React a framework? -> No, a UI library.
- What is a key? -> Unique stable identity for list items.
- Can we return two elements? -> Yes, wrapped in a Fragment or array.
- setState sync or async? -> Scheduled/batched; value updates on next render.
- Why className? -> `class` is a reserved word in JavaScript.
- What is a pure component? -> Same props/state gives same output, no side effects.
- What is reconciliation? -> Diffing old and new trees to update the DOM minimally.
- Controlled component? -> Form value driven by React state.
- Can hooks be used in class components? -> No.
- Which hook replaces componentDidMount? -> `useEffect` with `[]`.
- What is prop drilling? -> Passing props through many layers unnecessarily.
- What does `useCallback` return? -> A memoised function.
- What does `React.memo` do? -> Skips re-render when props are shallow-equal.
- Can `useEffect` be async? -> Not directly; call an async function inside.
- What is lazy loading? -> Loading code only when needed (`React.lazy` + `Suspense`).

Q159. Rapid-fire set 2

ANSWER

- Difference between `useMemo` and `useCallback`? -> value vs function.
- What is a synthetic event? -> React's cross-browser event wrapper.
- What are portals? -> Render children outside parent DOM hierarchy.
- Why keys should not be index? -> Reordering breaks identity and state.
- How do you pass data child to parent? -> Callback props.
- What is Context API? -> Built-in way to share data without prop drilling.
- What are Error Boundaries? -> Class components catching render errors.
- What is a HOC? -> Function taking a component and returning an enhanced one.
- What is Redux? -> Predictable global state container.
- What is a reducer? -> Pure function (state, action) => newState.
- `useRef` triggers re-render? -> No.
- What is hydration? -> Attaching React to server-rendered HTML.
- What is SSR? -> Rendering HTML on the server per request.
- What are React 18 highlights? -> Concurrent rendering, automatic batching, `useTransition`, `Suspense` on the server.
- What are React 19 highlights? -> Actions, `use()`, `useOptimistic`, `ref` as prop, Server Components stable.

Quick Reference Cheat Sheet

All Hooks at a Glance

Hook	Purpose	Re-renders on change?
<code>useState</code>	Local component state	Yes
<code>useReducer</code>	Complex state via reducer/actions	Yes
<code>useEffect</code>	Side effects after paint (fetch, subscribe)	n/a
<code>useLayoutEffect</code>	Sync effect before paint (measure DOM)	n/a
<code>useRef</code>	DOM access, mutable value that persists	No
<code>useContext</code>	Read context value	Yes when value changes
<code>useMemo</code>	Cache computed value	n/a
<code>useCallback</code>	Cache function reference	n/a
<code>useTransition</code>	Mark updates as non-urgent	Yes (isPending)
<code>useDeferredValue</code>	Lagging copy of a value	Yes
<code>useId</code>	Stable unique ids (SSR-safe)	No
<code>useSyncExternalStore</code>	Subscribe to external stores safely	Yes
<code>useImperativeHandle</code>	Customise ref exposed to parent	No
<code>useActionState</code> (19)	Form/action state + pending	Yes
<code>useOptimistic</code> (19)	Instant optimistic UI	Yes
<code>use</code> (19)	Read a promise or context	Suspends

Performance Checklist

- Measure first (Profiler, Lighthouse), optimise second.
- Colocate state; lift only when needed.
- Stable keys; never define components inside components.
- Memoise expensive values and callbacks passed to memoised children.
- Split code by route, lazy load heavy widgets, virtualise long lists.
- Optimise images, fonts and third-party scripts; use CDN and caching.

Common Mistakes Interviewers Notice

- Mutating state; using index as key; missing effect cleanup or dependencies.
- Storing derived data in state; too many effects; fetching without handling errors/race conditions.
- Using `div` instead of `button`; ignoring loading and empty states; huge components with no separation.

Final Pre-Interview Checklist

- I can explain Virtual DOM, reconciliation, keys and re-render triggers without notes.
- I can write `useState`, `useEffect`, `useRef`, `useReducer`, `useContext` examples from memory.
- I can explain stale closures and fix them.
- I can build a debounced search, todo, modal and infinite scroll in under 30 minutes.

- I can explain how I would optimise a slow React app.
- I know my projects deeply, and can explain every dependency and design decision.
- I know the basics of Redux Toolkit / Zustand, React Router, TanStack Query, testing and TypeScript.
- I know what is new in React 18 and 19.